

A) ¿Qué son los estados en React? ¿Cuáles son sus características y para qué sirve? Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo descrito, explicando el ejemplo.

El estado representa la memoria interna y privada de un componente en React, encargada de almacenar y controlar los datos que cambian a lo largo del tiempo como resultado de la interacción del usuario o de respuestas asíncronas de la red. A diferencia de las *props* —que son datos inmutables recibidos desde un componente padre de forma externa—, el estado es completamente mutable mediante su propia función de actualización y permanece encapsulado dentro del propio componente. Su principal característica y propósito es que, cada vez que el valor del estado sufre una modificación, React reacciona de forma automática disparando un nuevo renderizado del componente para sincronizar la interfaz visual con los datos más recientes de manera eficiente mediante el Virtual DOM.

```
import React, { useState } from 'react';

function Contador() {
  // Declaración del estado 'conteo' inicializado en 0 y su función actualizadora 'setConteo'
  const [conteo, setConteo] = useState(0);

  // Función manejadora para incrementar el valor del estado
  const incrementar = () => {
    setConteo(conteo + 1); // Notifica a React que el estado cambió para re-renderizar
  };

  return (
    <div>
      {/* Visualización del valor dinámico actual del estado */}
      <p>Has hecho clic {conteo} veces</p>
      {/* Botón que ejecuta la actualización del estado al hacer clic */}
      <button onClick={incrementar}>
        Incrementar
      </button>
    </div>
  );
}

export default Contador;
```

B) ¿Qué son los eventos en React? ¿Cuáles son sus características y para qué sirve? Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo descrito, explicando el ejemplo.

Los eventos en React son el mecanismo que permite capturar y responder a las distintas acciones realizadas por el usuario en la interfaz, tales como hacer clic en un botón, enviar formularios o escribir texto dentro de un campo de entrada. React implementa un sistema denominado SyntheticEvent, el cual envuelve los eventos nativos del navegador para garantizar un comportamiento idéntico y consistente en cualquier explorador web. Sus características clave incluyen el uso obligatorio de la convención de nomenclatura camelCase para los nombres de los manejadores (como onClick, onChange o onSubmit) y el paso directo de funciones como referencias entre llaves en JSX en lugar de cadenas de texto, permitiendo además evitar comportamientos predeterminados del navegador invocando explícitamente event.preventDefault().

```
import React, { useState } from 'react';

function FormularioTexto() {
  // Estado local para capturar el texto introducido
  const [texto, setTexto] = useState('');

  // Función manejadora del evento de cambio que recibe el SyntheticEvent (e)
  const manejarCambio = (e) => {
    // Se extrae el valor del input desde el evento y se actualiza el estado
    setTexto(e.target.value);
  };

  // Función manejadora para el evento de envío del formulario
  const manejarEnvio = (e) => {
    e.preventDefault(); // Previene la recarga por defecto de la página del navegador
    alert(`Texto enviado: ${texto}`);
  };

  return (
    // Asignación del evento onSubmit con sintaxis camelCase
    <form onSubmit={manejarEnvio}>
      {/* Asignación del evento onChange para detectar cada pulsación de tecla */}
      <input
        type="text"
        value={texto}
        onChange={manejarCambio}
        placeholder="Escribe algo aquí..."
      />
      <button type="submit">Enviar</button>
    </form>
  );
}

export default FormularioTexto;
```

C) ¿Qué son los Hook en React? ¿Cuáles son sus características y para qué sirve? ¿Cuáles son los principales Hook? Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo descrito, explicando el ejemplo.

Los Hooks son funciones especiales introducidas a partir de React 16.8 que permiten a los componentes funcionales conectarse al estado y a los ciclos de vida internos de React. Se caracterizan por seguir dos reglas fundamentales: sólo deben ser invocados en el nivel superior del componente y solo deben ejecutarse dentro de componentes funcionales de React o dentro de Hooks personalizados. Sirven para desacoplar y reutilizar la lógica con estado entre múltiples componentes de forma modular y limpia. Entre los principales Hooks provistos por React se destacan `useState` (para inicializar y modificar el estado local), `useEffect` (para gestionar efectos secundarios y ciclos de vida), `useContext` (para acceder al árbol global de contexto sin pasar props manualmente), `useRef` (para almacenar referencias mutables que no provocan re-renderizados o manipular nodos directos del DOM), `useReducer` (para lógica de estado más compleja) y `useMemo/useCallback` (para optimización del rendimiento mediante memorización).

```
import React, { useState, useEffect } from 'react';

function Temporizador() {
  // Hook useState: Inicializa y gestiona el estado local de segundos
  const [segundos, setSegundos] = useState(0);

  // Hook useEffect: Maneja efectos secundarios (suscripciones, temporizadores, ciclo de vida)
  useEffect(() => {
    // Configuración del intervalo al montarse el componente
    const intervalo = setInterval(() => {
      setSegundos((prevSegundos) => prevSegundos + 1);
    }, 1000);

    // Función de limpieza: Se ejecuta cuando el componente se desmonta del DOM
    return () => clearInterval(intervalo);
  }, []); // El arreglo vacío indica que el efecto solo se ejecuta al montarse y desmontarse

  return (
    <div>
      <h2>Temporizador Activo</h2>
      <p>Tiempo transcurrido: {segundos} segundos</p>
    </div>
  );
}

export default Temporizador;
```